

# Guía Práctica: Transacciones e Integridad en PostgreSQL

Para esta práctica simularemos operaciones cotidianas de un pequeño sistema de ventas.

Requisitos previos:

- Levantamos el contenedor con `docker compose start` ó `docker compose up -d`
- Abrimos el pgAdmin del contenedor, y luego registramos el servidor con los datos del dockerfile. Luego abrimos el **Query Tool** de la base de datos en pgAdmin.

## Parte 1: Simulando un fallo (sin transacciones)

Imaginemos el proceso de una compra web. El cliente paga (se le descuenta de su billetera) y luego se genera el pedido. ¿Qué pasa si ocurre un error entre un paso y el otro y no usamos transacciones?

**Ejecuta estas instrucciones UNA POR UNA:**

```
-- Consultamos el saldo del cliente 1 y el stock del articulo 1
SELECT id_cliente, saldo FROM billetera_clientes WHERE id_cliente = 1;
SELECT id_articulo, stock FROM articulos WHERE id_articulo = 1;

-- Como desea comprar el articulo 1 le restamos el saldo de su billetera
UPDATE billetera_clientes SET saldo = saldo - 5000 WHERE id_cliente = 1;

-- verificamos que el saldó se descontó
SELECT id_cliente, saldo FROM billetera_clientes WHERE id_cliente = 1;

-- descontamos el stock del articulo
UPDATE articulos SET stock = stock - 1 WHERE id_articulo = 1;

-- insertamos el pedido, pero nos faltó validar algún dato
-- Sólo simula un fallo, que también podría ser corte de luz, pérdida de conexión, etc.
```

```
INSERT INTO pedidos (id_cliente, id_articulo, fecha, cantidad,
importe)
VALUES (1, 1, 'fecha errónea o no validada', 1, 5000);
```

¿Qué pasó? ¿Cómo podemos evitarlo?

- La base quedó en estado inconsistente
- no existió atomicidad para el conjunto de operaciones
- la solución es usar transacciones para definir un conjunto de operaciones que deben ejecutarse de forma atómica

```
-- dejamos la base en estado consistente: devolvemos la plata y el
stock
UPDATE billetera_clientes SET saldo = saldo + 5000 WHERE id_cliente
= 1;
UPDATE articulos SET stock = stock + 1 WHERE id_articulo = 1;
```

## Parte 2: Atomicidad con BEGIN y COMMIT

Ahora haremos una venta real y completa, pero le diremos a PostgreSQL que trate a todas las instrucciones como **una sola unidad indivisible** usando una transacción.

**Selecciona TODO este bloque y ejecútalo junto:**

```
BEGIN;

-- 1. Cobramos el importe
UPDATE billetera_clientes SET saldo = saldo - 5000.00 WHERE
id_cliente = 1;

-- 2. Descontamos el stock del artículo
UPDATE articulos SET stock = stock - 1 WHERE id_articulo = 1;

-- 3. Registramos el pedido exitosamente
INSERT INTO pedidos (id_cliente, id_articulo, fecha, cantidad,
importe, estado)
VALUES (1, 1, CURRENT_DATE, 1, 5000.00, 'PAGADO');

COMMIT; -- Guardado permanente
```

**Análisis:** Todas las operaciones dependían entre sí y se ejecutaron juntas. La transacción pasó a estado **Confirmada**. El sistema es 100% consistente.

## Parte 3: Comprobando el Aislamiento (Isolation)

¿Qué pasa si un cliente está mirando el catálogo exactamente en el mismo milisegundo en que un administrador está actualizando el precio de un artículo?

Para probar esto, **abre una SEGUNDA PESTAÑA de Query Tool en pgAdmin.**

- **Pestaña 1:** Será el "Administrador".
- **Pestaña 2:** Será el "Cliente mirando la tienda web".

**Paso a paso:**

**(Pestaña 1 - Admin):** Abre una transacción y cambia el precio, *pero no hagas COMMIT todavía.*

```
BEGIN;
```

```
UPDATE articulos SET precio_unitario = 99999.99 WHERE id_articulo = 1;
```

```
SELECT precio_unitario FROM articulos WHERE id_articulo = 1;
```

(Verás el precio carísimo en **99999.99**.)

**(Pestaña 2 - Cliente):** Ejecuta el mismo SELECT.

```
SELECT precio_unitario FROM articulos WHERE id_articulo = 1;
```

**¡Sorpresa! El cliente sigue viendo el precio viejo original porque la otra transacción no finalizó.**

**(Pestaña 1 - Admin):** Ahora sí, cierra la transacción confirmando cambios.

```
COMMIT;
```

**(Pestaña 2 - Cliente):** Vuelve a ejecutar el SELECT.

```
SELECT precio_unitario FROM articulos WHERE id_articulo = 1;
```

Ahora sí verás el nuevo precio actualizado.

Esto demuestra el **Aislamiento**. Las transacciones no muestran sus cambios intermedios al resto del sistema hasta estar confirmadas (COMMIT).

## Parte 4: Transacciones vs. Integridad Referencial

La tabla billetera\_clientes tiene una restricción vital: CHECK (saldo >= 0). Un cliente no puede gastar dinero que no tiene.

Veamos qué hace Postgres si intentamos violar esta regla dentro de una transacción en curso (simulando que el cliente 1 que solo tiene \$45000 intenta comprar el artículo 1 que quedó con el precio unitario establecido en 99999.99).

Ejecuta **línea por línea**:

BEGIN;

-- 1. Insertamos el pedido de \$99999.99 (Esto funciona bien, el registro se crea temporalmente)

```
INSERT INTO pedidos (id_cliente, id_articulo, fecha, cantidad, importe)
```

```
VALUES (1, 1, CURRENT_DATE, 1, 99999.99);
```

-- 2. Intentamos cobrarle los \$99999.99 a la billetera (Rompe el CHECK porque el saldo quedaría en negativo)

```
UPDATE billetera_clientes SET saldo = saldo - 99999.99 WHERE id_cliente = 1;
```

ERROR: new row for relation "billetera\_clientes" violates check constraint "billetera\_clientes\_saldo\_check".

Failing row contains (1, -54999.99).

-- 3. Terminamos la transacción obligatoriamente

ROLLBACK;

Al violar la restricción CHECK del saldo en el Paso 2, la transacción entró en **Estado Fallida**.

Una vez que la transacción falla, el motor se "bloquea" por seguridad, ignora cualquier instrucción posterior (como la auditoría del Paso 3) y nos obliga a ejecutar ROLLBACK.

Al hacer el Rollback, el pedido del Paso 1 que sí era válido se borra automáticamente, asegurando que no queden pedidos registrados sin haber sido cobrados.

**Tip para los prácticos:** Si en algún momento "rompemos" los datos o borramos cosas sin querer, tenemos la ventaja de estar ejecutando la base en docker con el script de inicialización, así que sólo debemos ir a la terminal y ejecutar `docker compose down -v` seguido de `docker compose up -d`.